Contents

1. Introduction 2
2. Project proposal and problem statement 2
3. Dataset description 2
4. Data preprocessing and visualisation 4
5. Implementation and evaluation 5
5.1 Multilayer perceptron classifier 6
5.2 Random Forest ensemble model 7
5.3 Convolutional neural network 8
5.4 Improved CNN experiment 10
5.5 Clustering with MiniBatch K-Means 12
5.6 Results comparison 14
6. Ethical and social impact 15
7. Conclusion 16
References 16

1. Introduction

The project uses machine learning techniques and deep learning techniques for classification of images of animals into ten different classes of species. This involves the classification of an image into one of ten species using both supervised and unsupervised techniques of learning. The problem statement that we have chosen has practical significance, and can be applied in areas like wildlife management, animal shelter management, educational websites, and agriculture support services. Image classification is expected to help in saving manual effort for sorting purposes and faster decision making in case a lot of images need to be evaluated.

In this project, the data was acquired from a publicly available dataset. After data preprocessing and visualisation, several different machine learning algorithms were used, such as multilayer perceptron, ensemble algorithm, CNN, and clustering. The project was carried out in Google Colab with the help of various libraries like TensorFlow/Keras and scikit-learn.

2. Project proposal and problem statement

The selected use case relates to the automatic identification of animal species based on imagery. Formulating the problem, we assume that the task is to identify the animal class corresponding to the image. Thus, the problem is multi-class image classification, as there will be ten classes to choose from. The problem can be solved using artificial intelligence due to its capability to automatically handle variability among animals in terms of pose, colors, illumination, background, size, and camera used.

There are four methods used to solve the problem. As the baseline approaches, a multilayer perceptron classifier will be applied using flattened pixels of the image as features. Moreover, we can apply Random Forest to those features to have a fair comparison. Convolutional neural networks (CNN) will be used due to their ability to learn spatial properties of images, including edges, textures, and shapes. Furthermore, Minibatch K-Means will be considered since it allows analysing the cluster structure of images when the labels are absent in the process of training. In addition, another CNN will be implemented using data augmentation and early stopping as regularisation techniques.

3. Dataset description

The Animals-10 dataset from Kaggle has been used as the dataset for the study. It consists of a dataset with 10 different categories of animals with labeled classes, multiple categories, over 100 samples, and very high dimensionality as images must be saved as arrays (Corrado, n.d.). In the current analysis, the downloaded dataset consists of 26,179 image files belonging to 10 different categories.

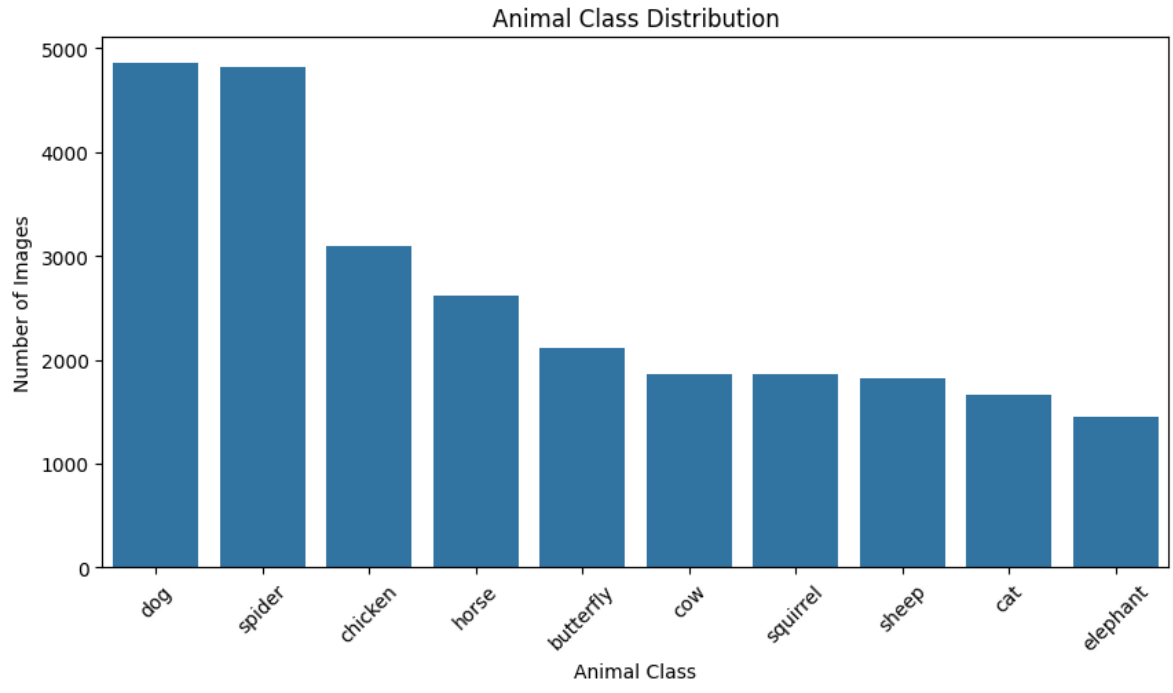
Initially, the folder names were written in Italian, which were modified within the notebook as follows: cane(dog), cavallo(horse), Elefante(elephant), farfalle(butterfly), Gallina(chicken), Gatto(cat), mucca(cow), Pecora(sheep), Ragno(spider) and scoiattolo(squirrel).

Table 1. Original class distribution

Class	Number of images
dog	4863
spider	4821
chicken	3098
horse	2623
butterfly	2112
cow	1866

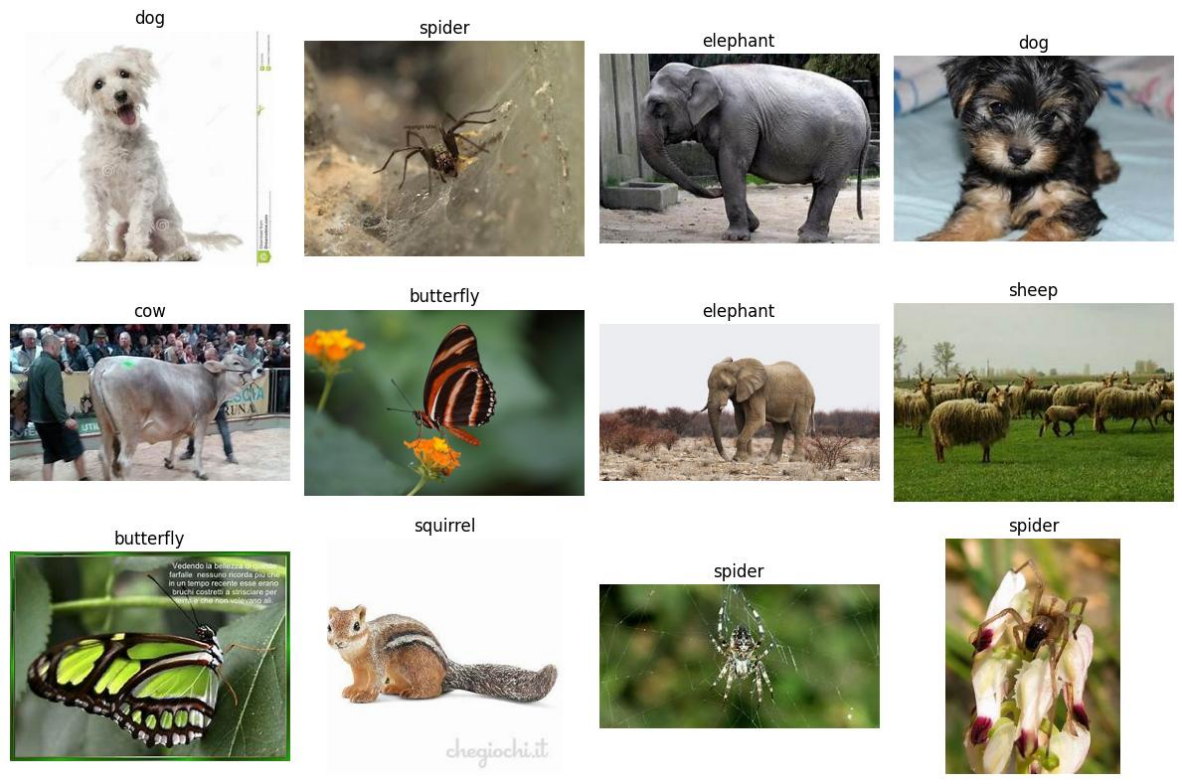
Class	Number of images
squirrel	1862
sheep	1820
cat	1668
elephant	1446

Figure 1. Original class distribution chart



The above chart helps in demonstrating data imbalance.

Figure 2. Sample images from the dataset



This figure is useful because it shows the variation in the dataset, such as different backgrounds, image angles, object sizes and image quality.

4. Data preprocessing and visualisation

Missing values, class balancing, dimensions of images, and labeling were considered during the preprocessing stage. There were no null values found in either image_path or label. There was an initial bias in the classes, which were dominated by more classes belonging to dogs and spiders as compared to elephants. The bias was resolved by selecting 1,000 instances of each class, totaling 10,000 images.

These images varied in size and needed to be normalized prior to building the models. All images involved in MLP model creation, random forest model creation, convolutional neural network (CNN), improved CNN, and cluster models were resized to 32*32 pixels with three channels. Pixel values were normalised such that their values ranged between 0 and 1. Images for all models except the cluster models were flattened into vectors with 3,072 attributes. For the CNN models, the images remained in 32 by 32 by 3 formats so the models could use spatial structure.

The labels in the classes were represented using integers starting from 0 to 9 and were also represented using one-hot encoding when needed for training purposes. A balanced dataset is split between training, validation and test sets at a ratio of 70%, 15% and 15%.

Table 2. Final data split

Subset	Number of images	Purpose
Training set	7000	Model fitting
Validation set	1500	Model tuning and monitoring

Subset	Number of images	Purpose
Test set	1500	Final model evaluation

Figure 3. Image width and height distribution

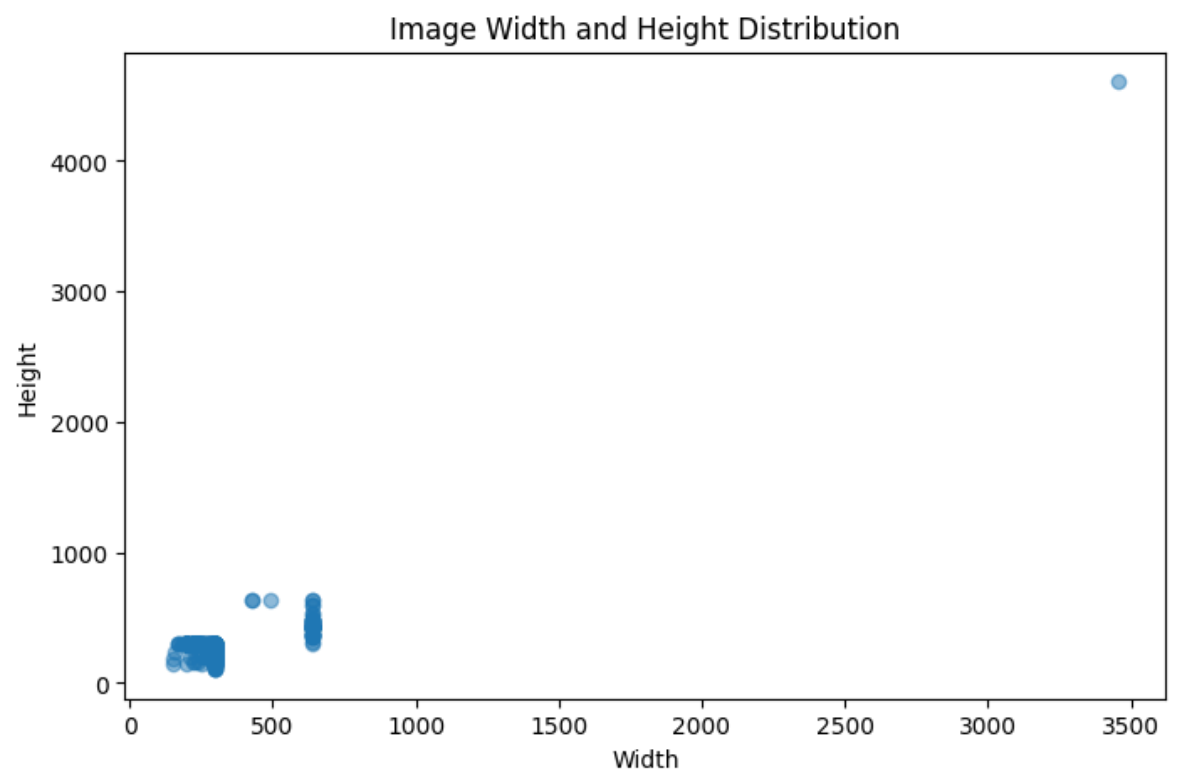


Figure 3. Image width and height distribution in a sample of the Animal-10 dataset.

It supports your preprocessing section because it shows that the images are of different sizes. That justifies resizing all images to 32 × 32 before training the MLP, Random Forest, CNN, and K-Means models.

The chosen features are relevant because they represent information regarding colors, edges, texture, and shape using RGB pixel values. Such information can be used in the classification process. The importance of the labels is that they stand for the category of animal appearing in the picture.

Two main shortcomings of the data set can be noted. Firstly, there exists class imbalance among the categories within the raw data set. In case it is not addressed, class imbalance has a high likelihood of resulting in biased models favoring common classes. Secondly, there exist differences in background, image quality, size, pose, and illumination among other aspects. This poses a challenge to the animal classification task and reduces the generalisability of the model. Another limitation is the fact that images come from internet sources.

5. Implementation and evaluation

The accuracy, classification report, and confusion matrix were used to analyse the performance of supervised methods. Accuracy referred to the ratio of correct classifications. Precision, recall, and F1 score measures were used to explain the performance per class. The confusion matrix helped in detecting inter-

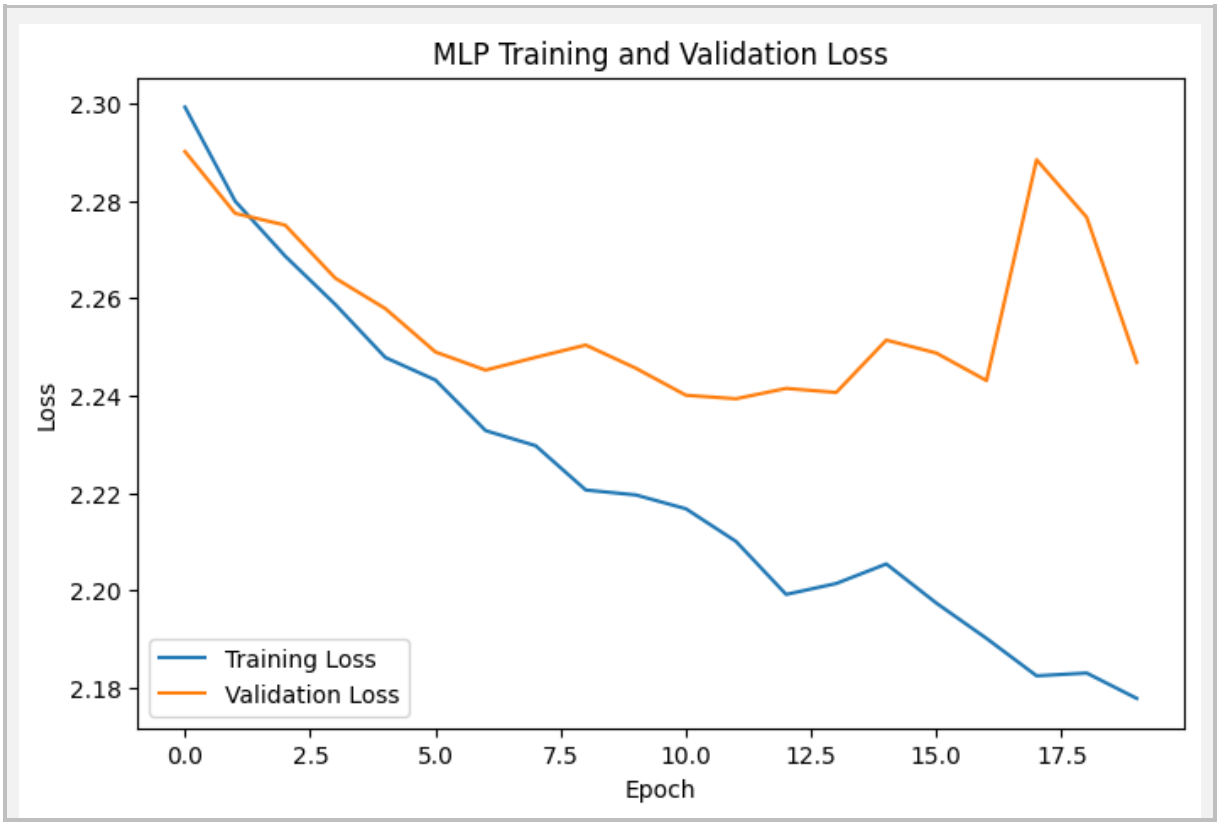
class confusion for animals. Clustering technique was analysed based on clustering accuracy, adjusted rand index, normalised mutual information, inertia, and silhouette measure.

5.1 Multilayer perceptron classifier

As for the MLP architecture used, TensorFlow/Keras was used with a hidden layer architecture of 15, 25, 25, and 20 neurons respectively. The ReLU activation was used on the hidden layers because it is the most popular in neural networks as an introduction of nonlinearity. Output neurons in the model were 10, using Softmax function activation. The model used the Adam optimiser and a learning rate of 0.001 for the purpose of optimisation. The cost function used in the process was categorical cross-entropy, and it was trained for 20 epochs with a batch size of 64. About the used architecture, the use of the Keras Sequential API is justified by the fact that it consists of a layer-by-layer combination of tensors (one input and one output tensors) (TensorFlow Developers, 2025).

Test accuracy for MLP reached 0.1580. While the test score is not impressive, it can act as a good baseline against which the model can be improved upon. The biggest disadvantage lies in flattening the 32 by 32 images since this removes the context between the neighboring pixels and reduces the capacity of learning image structures like edges and textures.

Figure 4. MLP training and validation accuracy and loss



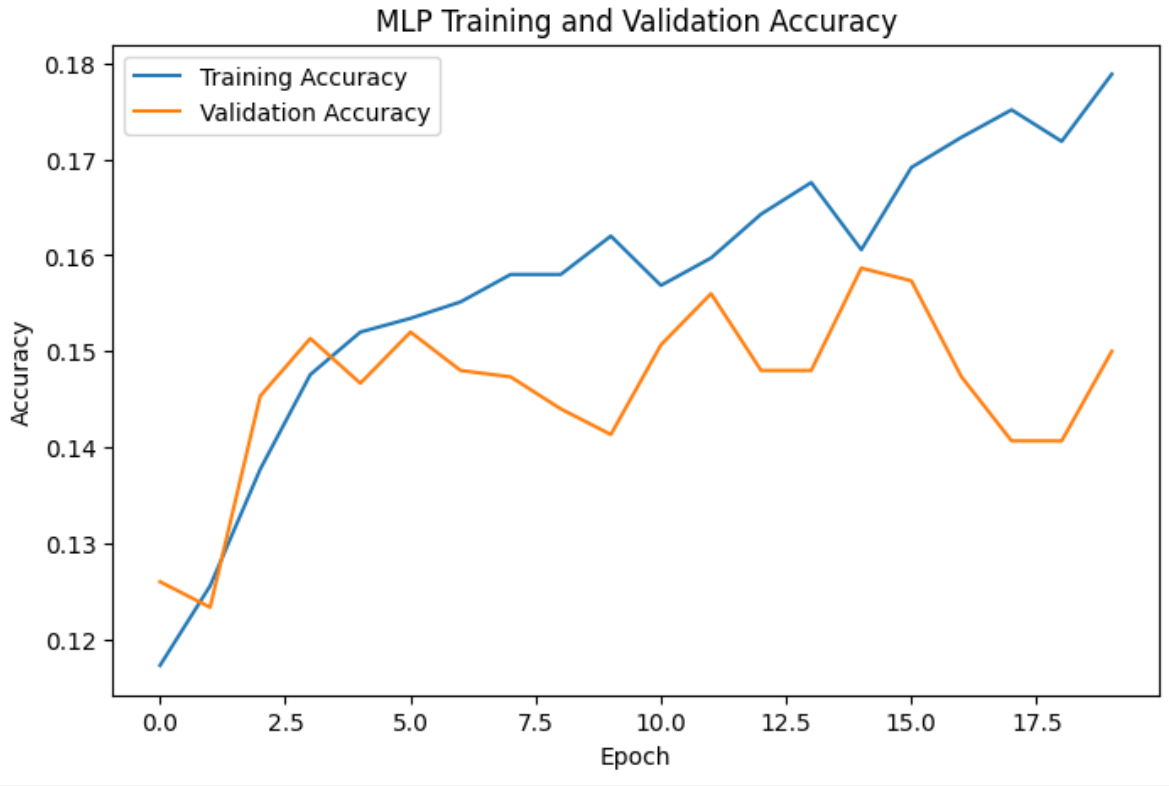


Figure 4. Training and validation accuracy and loss curves for the MLP classifier using flattened 32 × 32 image pixels.

5.2 Random Forest ensemble model

A Random Forest was used as the classifier in the ensemble learning approach through scikit-learn. The Random Forest consists of an aggregation of many decision trees that average their predictions to improve accuracy and decrease dependence on one specific tree (Scikit-learn developers, n.d.a). This setting involved the use of 100 trees, a maximum depth of 25, n_jobs parameter of -1 to speed up training and a constant value for the random state to increase reproducibility. The feature set in use had 3,072 features like in the case of the MLP model to facilitate comparability. The accuracy rate of the Random Forest model in validating the model was 0.3520 while that during testing was 0.3567, thus performing better than the MLP model because the former model found more patterns from the data. From the results of the classification report, there appeared to be superior performance in classifying the butterfly, spider, sheep, horse, and elephant compared to the dog and chicken.

Figure 5. Random Forest confusion matrix

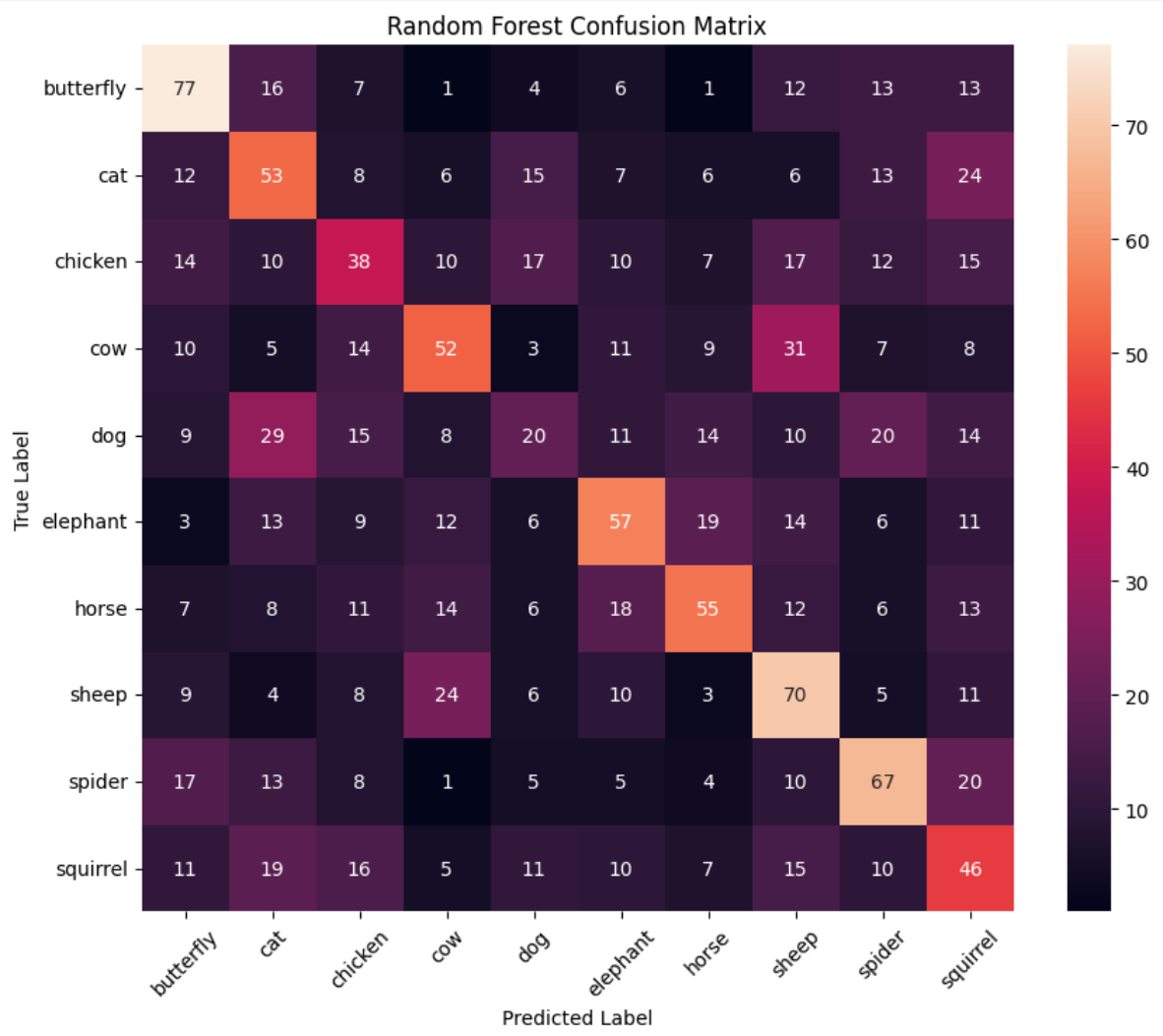


Figure 5. Random Forest confusion matrix showing the predicted and true labels for the 10 animal classes on the test set.

5.3 Convolutional neural network

CNN network model was created using TensorFlow/Keras library. Four convolution layers were used within the CNN architecture based on the course requirements. Each of the convolution layers consisted of 32, 64, 128, and 128 filters, respectively, which were then normalised via the batch and max pooling processes. Flattened data passed through a dropout process and dense layer of 256 neurons and ended up with 10 neuron softmax layer as an output. Dropout was used to reduce the problem of overfitting (Srivastava et al., 2014). The performance of the model was better than those of other networks. The performance of the CNN was 0.5420, which was as expected since the CNN architecture preserves the two-dimensional data of an image where features like texture, shape, and edges are learned. Overfitting was observed during training where the training accuracy was 0.91 while the validation accuracy was about 0.55.

Figure 6. CNN training and validation accuracy and loss

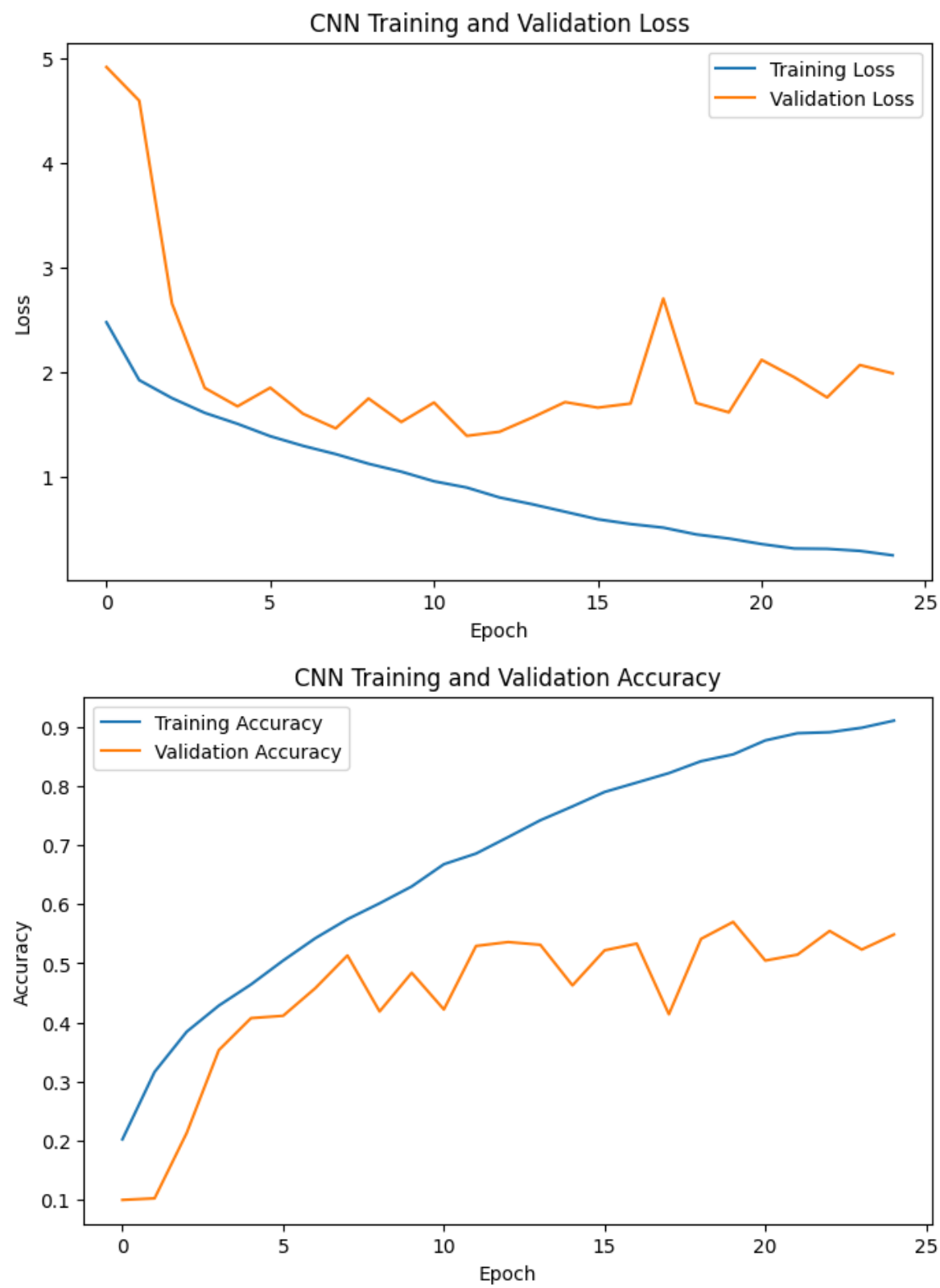


Figure 6. Training and validation accuracy and loss curves for the CNN model using 32 × 32 animal images.

Figure 7. CNN confusion matrix

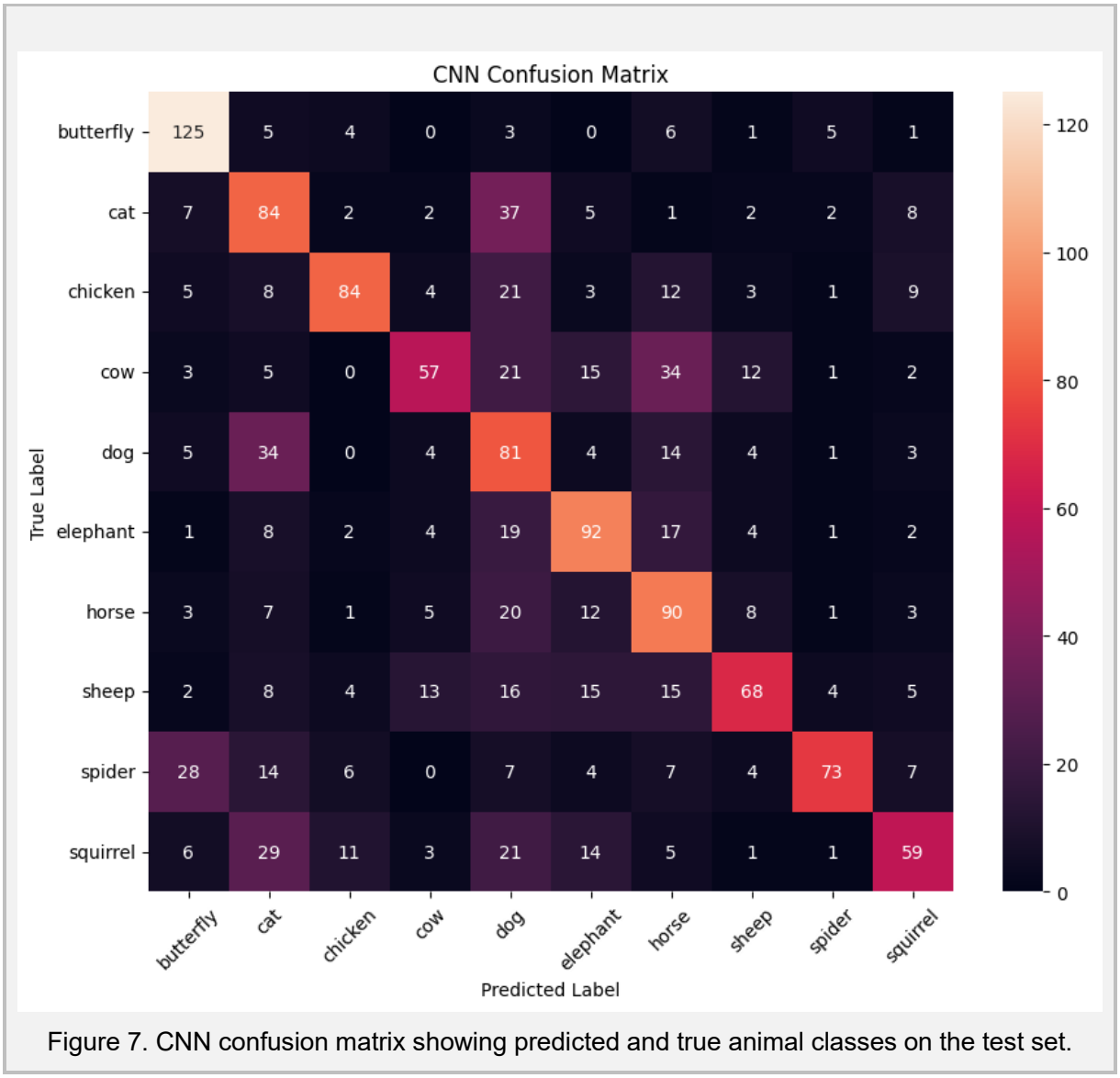


Figure 7. CNN confusion matrix showing predicted and true animal classes on the test set.

5.4 Improved CNN experiment

An enhanced CNN experiment was proposed to serve as an improved extension to the CNN. Some of the improvements added include data augmentation, early stopping, learning rate reduction, batch normalisation, and dropout. For data augmentation, the techniques used were horizontal flipping, rotating, and zooming. All these techniques were implemented to improve the problem of overfitting and generalise the model. The ReduceLROnPlateau technique was applied since it is easier for a model to perform better by reducing the learning rate once validation starts slowing down (TensorFlow Developers, 2024).

The enhanced CNN produced test accuracy of 0.4753 compared to 0.5420 accuracy for the original CNN. The two curves (training and validation curves) showed a similarity in the enhanced CNN compared to that of the original CNN. Overfitting seems to be less prevalent here. This might be since the images in the database are small (32 x 32). The data augmentation process on such small images will strip away some vital visual cues from the images.

Figure 8. Improved CNN training and validation accuracy and loss

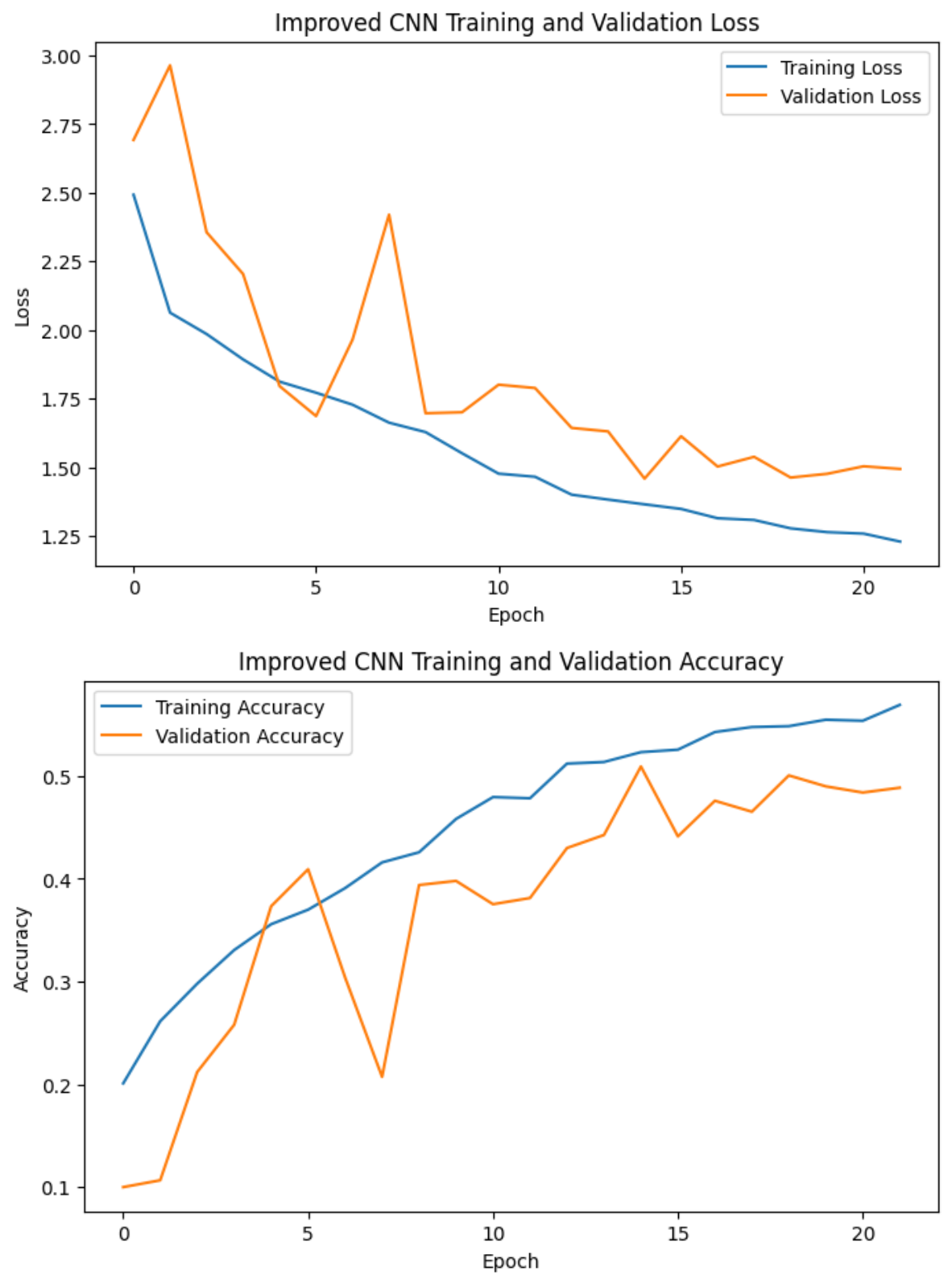


Figure 8. Improved CNN training and validation accuracy and loss curves.

Figure 9. Improved CNN confusion matrix

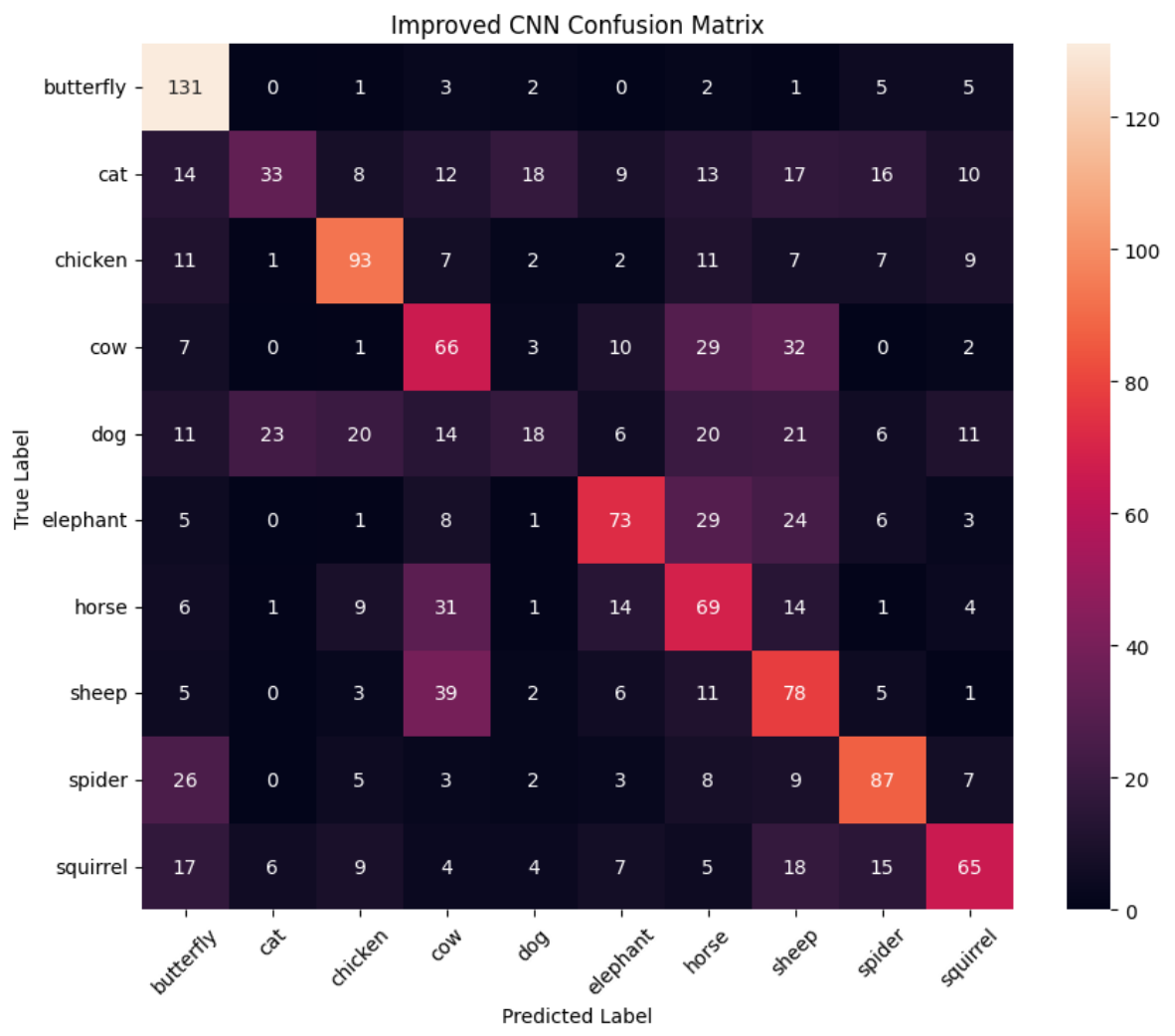


Figure 9. Improved CNN confusion matrix showing predicted and true animal classes on the test set.

5.5 Clustering with MiniBatch K-Means

Clustering was performed using mini-batch K-Means, which provides computational benefits compared to regular K-Means algorithm, using scikit-learn library (Scikit-learn developers, n.d.b). Prior to clustering, a PCA transformation was performed that reduced dimensionality of the feature vector from 3,072 to 100 and retained about 77.15% variance of the dataset.

Inertia and the silhouette coefficient were used for evaluating the models with clusters numbering between 2 and 15. The highest silhouette score was obtained at k=2, meaning that the clustering algorithm detected general visual clusters and not the ten unique classes in the dataset. Since the number of classes in the dataset is ten, a ten-cluster model was developed using this algorithm and compared with the true labels using the Hungarian method. In this case, the accuracy for the ten-cluster model was found to be 0.1755, while the Adjusted Rand Index and Normalised Mutual Information were found to be 0.0178 and 0.0427 respectively. The values being close to zero imply that the match between the cluster labels and the actual labels is like random matching (Scikit-learn developers, n.d.c).

Figure 10. K-Means elbow and silhouette plots

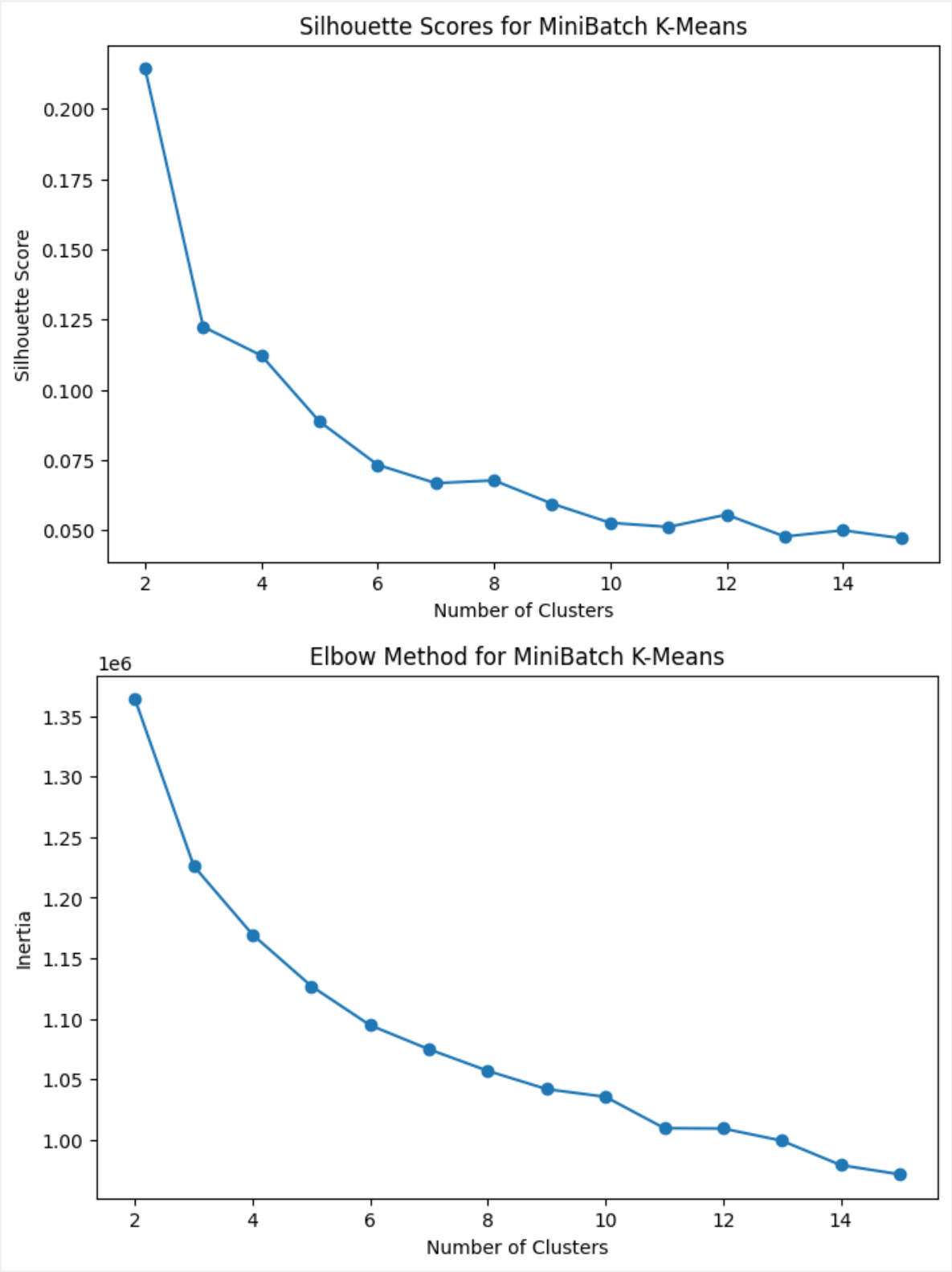


Figure 10. Elbow method and silhouette score plots used to estimate the suitable number of clusters for MiniBatch K-Means.

Figure 11. MiniBatch K-Means confusion matrix

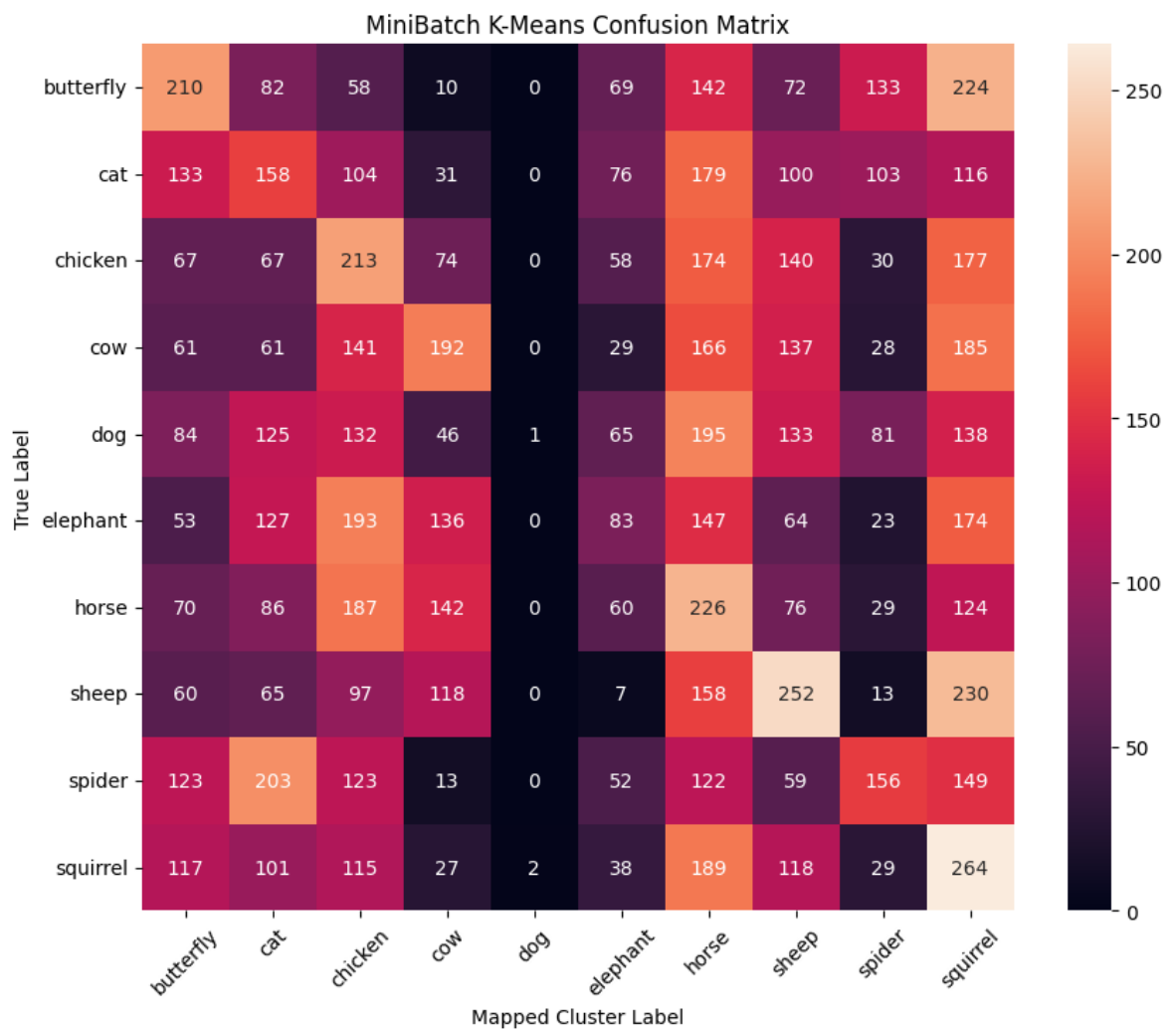


Figure 11. Confusion matrix for MiniBatch K-Means clustering after mapping clusters to the true animal classes.

5.6 Results comparison

Table 3. Supervised model accuracy comparison

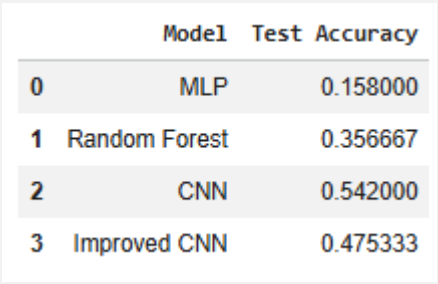
Model	Test accuracy	Interpretation
MLP	0.1580	Weak baseline using flattened pixels
Random Forest	0.3567	Improved result from ensemble learning
CNN	0.5420	Best required supervised model
Improved CNN	0.4753	Reduced overfitting, but lower test accuracy

Table 4. Clustering results

Metric	Value
Best k by silhouette score	2
Clusters used for class comparison	10
Clustering accuracy	0.1755
Adjusted Rand Index	0.0178
Normalised Mutual Information	0.0427

CNN provided the best results in supervised testing. That indicates the thesis statement according to which CNN is better for performing image classification due to its capability of keeping the spatial data intact while learning features in images. The Random Forest proved better than MLP; nevertheless, all the methods were limited by using flattened features of the images. Clustering did not deliver good results since K-Means did not rely on the labels while learning and clustered images based on pixel-based similarities, rather than on notions related to animals' species.

Figure 12. Final model comparison table or chart



	Model	Test Accuracy
0	MLP	0.158000
1	Random Forest	0.356667
2	CNN	0.542000
3	Improved CNN	0.475333

Figure 12. Final comparison table showing the test accuracies of the MLP, Random Forest, CNN, and improved CNN models.

6. Ethical and social impact

Classification of animal species can generate social benefits when done in a responsible way. Such applications may range from biodiversity control to a reduction in manual labor in animal shelters, educational systems in identifying animal pictures, farms or conservationists in picture sorting. Classification of animals could assist in analysing quickly huge amounts of camera trap pictures that require inspection. This will be possible when the technology is used in an appropriate manner and with proper human supervision.

The first and most serious risk that is involved in the use of this technology is wrong predictions, which might cause bad outcomes in situations like conservation, tracking the presence of diseases, or on the farm. In such a case, the classification might have to help the person make decisions but never take the place of experts.

The other problem related to classification is bias. The problem results from imbalance in the classes, wherein some classes are more represented than others. Moreover, there are also class differences that caused the model to prefer some classes. In this case, although this problem was resolved by ensuring equal treatment of all classes, the training set also got smaller accordingly.

In relation to privacy and security issues, it can be expected that videos of wildlife taken from cameras will contain pictures of people, possessions, geographical locations, or rare species of animals. These should be protected and used only for sharing purposes. What's more, it is also vital to make sure that sensitive data is not shared with people who may harm animals.

7. Conclusion

In this project, the development of multiple machine learning models for animal species classification was performed. The dataset provided by Kaggle was used in training the models. After data preparation which involved loading, cleaning, translating labels to English, balancing to 1,000 images per label, resizing, normalising, and splitting into train, validate, and test data, the baseline accuracy of the multilayer perceptron model was surprisingly low at 0.1580. The Random Forest model gave an accuracy of 0.3567. The highest test accuracy of 0.5420 was obtained by the CNN model. Another variant of the CNN, aimed at decreasing overfitting, resulted in a decreased test accuracy equal to 0.4753. As far as the MiniBatch K-Means clustering algorithm is concerned, its test accuracy score was quite poor, being equal to 0.1755.

It may be concluded from the obtained scores that CNN models perform better when working with images rather than flat arrays consisting of pixel values. In the future, performance can be improved using bigger pictures, more sophisticated data augmentations, and hyperparameters' tuning among other strategies.

References

Corrado, A. (n.d.) 'Animals-10'. Kaggle. Available at: https://www.kaggle.com/datasets/alessiocorrado99/animals10 (Accessed: 28 May 2026).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825-2830. Available at: https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html (Accessed: 1 June 2026).

Scikit-learn developers (n.d.a) 'RandomForestClassifier'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html (Accessed: 28 May 2026).

Scikit-learn developers (n.d.b) 'MiniBatchKMeans'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.MiniBatchKMeans.html (Accessed: 29 May 2026).

Scikit-learn developers (n.d.c) 'Adjusted Rand score'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html (Accessed: 29 May 2026).

Scikit-learn developers (n.d.d) 'Normalised mutual information score'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.normalized_mutual_info_score.html (Accessed: 30 May 2026).

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: A simple way to prevent neural networks from overfitting', Journal of Machine Learning Research, 15, pp. 1929-1958. Available at: https://jmlr.org/papers/v15/srivastava14a.html (Accessed: 31 May 2026).

TensorFlow Developers (2024) 'ReduceLROnPlateau'. TensorFlow. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/ReduceLROnPlateau (Accessed: 1 June 2026).

TensorFlow Developers (2025) 'The Sequential model'. TensorFlow. Available at: https://www.tensorflow.org/guide/keras/sequential_model (Accessed: 1 June 2026).